

T02 — Paradigma ad Oggetti e Modellazione UML

Astrazione, Incapsulamento, Modularità, Gerarchia e Sintassi dei Class Diagram

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo la transizione teorico-metodologica dal paradigma procedurale a quello ad oggetti, i concetti cardine dell'OOP e la modellazione grafica tramite **UML** ([T02 - Object-Orientations and UML.pdf](#)).

0.0.1 1. Teoria: Concetti Teorici dalle Slide

A. La Tassonomia dei Linguaggi e il Bisogno di Astrazione (Slide 1–13)

- **Nessun linguaggio universale:** citando Edsger W. Dijkstra («*Computer Science is no more about computers than Astronomy is about telescopes*»), il docente introduce il principio del **Golden Hammer** («*If all you have is a hammer, everything looks like a nail*»): non esiste un linguaggio perfetto per tutto.
- **Evoluzione storica degli obiettivi:**
 - **Anni '50/'60:** focus sull'**efficienza del codice macchina**. L'hardware costava milioni e i programmatori poco: l'obiettivo era “tenere occupata la macchina” con linguaggi vicini al silicio.
 - **Oggi:** focus sull'**efficienza dello sviluppo del software**. L'hardware costa poco e i programmatori costano molto: l'obiettivo è “tenere produttivo il programmatore”, abbattendo complessità, bug e costi di manutenzione tramite l'**astrazione**.
- **I Tre Paradigmi Fondamentali:**
 1. **Imperativo / Procedurale** (C, Pascal, Fortran, COBOL):
 - Basato sul concetto di **stato** (celle di memoria che memorizzano valori) e **comandi sequenziali** che mutano tale stato (assegnamenti).
 - Riutilizzo del codice tramite procedure/funzioni che operano su dati esterni.
 2. **Dichiarativo:**
 - **Funzionale** (Haskell, Lisp): il programma è la valutazione di un'espressione matematica pura, basata su applicazione di funzioni e ricorsione (es. `add x y = x + y`). Assenza o minimizzazione degli effetti collaterali (*side effects*).
 - **Logico** (Prolog): il programma definisce relazioni, fatti e regole di deduzione logica; l'esecuzione è una risoluzione/unificazione guidata da query (es. `playsAirGuitar(sam):- listens2Music(sam).`).
 3. **Object-Oriented (OOP)** (Smalltalk, Java, C++):
 - Unisce il nucleo imperativo (gli oggetti contengono uno stato interno) a tratti dichiarativi (gli oggetti espongono comportamenti invocabili senza conoscerne l'algoritmo interno).
 - Basato su **istanze** che comunicano tramite **passaggio di messaggi**.

- **Lo Spettro dell'Astrazione e il Trade-off di Overhead:**

Machine Code → Assembly → Procedurale → Object-Oriented → Funzionale

– Più si sale nell'astrazione:

- * **Programmer Overhead** ↘ (**minimo**): meno tempo perso a gestire registri, puntatori e memoria manuale.
- * **Compiler/Runtime Overhead** ↗ (**massimo**): il compilatore/JVM deve effettuare controlli di tipo a runtime, garbage collection e risoluzione dinamica delle chiamate (*dynamic dispatch*).
- **Linea temporale:** dal 1949 (Assembly), Fortran (1957), C (1972), C++ (1983) fino all'avvento di Java (1995), C# (2000) e linguaggi moderni multiparadigma (Swift 2014).

B. La Crisi del Procedurale e la Nascita degli Oggetti (Slide 14–23) Il prof. Pasqua presenta il caso di studio di un programma in C che manipola un vettore globale:

```

1 int vect[20];
2 void sort() { /* ordina vect */ }
3 int search(int n) { /* cerca in vect */ }
4 void init() { /* inizializza vect */ }
5 int i;
6
7 void main() {
8     init();
9     sort();
10    search(13);
11 }
```

I 4 Difetti Letali dell'Approccio Procedurale evidenziati a lezione:

1. **Relazione Implicita Dati-Funzioni:** vect e le funzioni sort(), search() sono entità slegate a livello di linguaggio. Non esiste un tipo autonomo "Vettore" che protegga se stesso.
2. **Nessun Controllo dei Confini (Out-of-Bounds):** un ciclo come for (i=0; i<=21; i++)vect[i]=0; scrive in memoria oltre i 20 elementi, corrompendo altre variabili o lo stack senza alcun errore del compilatore (*buffer overflow*).
3. **Nessuna Garanzia sull'Inizializzazione:** se il programmatore dimentica di invocare init(), le funzioni sort() e search() lavorano su memoria sporca non inizializzata.
4. **Perdita di Confidenzialità e Integrità:** vect è accessibile in lettura/scrittura da *qualsiasi* funzione del programma. Su sistemi reali questo genera il cosiddetto **Spaghetti Code**.

- **La Regola delle Linee di Codice (LoC):** - Per script < 30 LoC: lo stile procedurale è rapido e facile da mantenere. - Per progetti > 1000 LoC con team concorrenti: lo stile procedurale collassa; solo l'OOP permette di modularizzare garantendo la manutenibilità.

- **La Soluzione a Oggetti:** - Unire nello stesso modulo sia i dati (**attributi/campi**) sia le operazioni su di essi (**metodi**). - L'oggetto diventa l'unico responsabile della consistenza del proprio stato.

C. Modellazione UML e Concetti Fondamentali di OOP (Slide 24–36)

- **UML (Unified Modeling Language):** linguaggio grafico standard per specificare e documentare sistemi ad oggetti.
- **Differenza Ontologica tra Classe e Oggetto:**
 - **Classe:** è il *tipo* (blueprint / modello astratto). Definisce gli attributi e i metodi. Non occupa memoria nello heap per i dati dell'istanza finché non viene istanziata.
 - **Oggetto:** è l'**istanza concreta** nello **Heap**. Possiede:
 1. **Stato:** i valori correnti associati ai suoi attributi.

- 2. **Comportamento:** i metodi che può eseguire.
 - 3. **Identità:** indirizzo di memoria univoco che lo distingue da tutti gli altri oggetti, anche se aventi lo stesso identico stato!
- **Rappresentazione grafica UML:**
 - **Classe (Rettangolo a 3 sezioni):**
 1. Nome della classe (es. Car).
 2. Attributi con tipo (es. `manufacturer : string, model : string`).
 3. Metodi con segnatura (es. `canFly(): boolean, setFrameNumber(num : uint)`).
 - **Oggetto / Istanza (Rettangolo con nome sottolineato):**
 - * Es. `Car1: Car` CON `manufacturer = DeLorean, model = DMC-12`.
 - **Interazione a Passaggio di Messaggi (*Message Passing*):**
 - Gli oggetti non leggono né modificano la memoria altrui direttamente: inviano richieste di servizio (*messaggi*).
 - L'**interfaccia** dell'oggetto è l'insieme di messaggi che esso accetta. Se invio un messaggio non previsto, si genera un errore di compilazione.
 - **Visibilità:**
 - `+` (`public`): accessibile da qualsiasi classe.
 - `-` (`private`): accessibile esclusivamente dall'interno della classe stessa.
 - **Incapsulamento vs Information Hiding:**
 - **Incapsulamento:** impacchettare dati e codice in una singola unità logica (la classe).
 - **Information Hiding:** nascondere i dettagli implementativi (rendendo i campi `private`) ed esporre solo un'interfaccia controllata (metodi `public`).
 - *Vantaggio chiave:* se cambiamo la struttura dati interna (es. da array a tabella hash), il codice esterno che usa i metodi pubblici non subisce alcun impatto (*zero refactoring esterno*).
 - **Associazioni UML:**
 - **Associazione standard:** linea continua con cardinalità (es. 1 a 0..n tra Insurance e Vehicle).
 - **Ereditarietà (Generalizzazione):** freccia con punta triangolare vuota orientata verso la super-classe (Car estende Vehicle, Truck estende Vehicle).
 - **I 4 Pilastri dell'OOP:**
 1. **Incapsulamento (*Encapsulation*):** bundling e protezione dello stato interno.
 2. **Astrazione (*Abstraction*):** separazione netta tra cosa un componente fa (interfaccia pubblica) e come lo fa (implementazione privata).
 3. **Ereditarietà (*Inheritance*):** meccanismo di riuso e specializzazione gerarchica di proprietà e comportamenti.
 4. **Polimorfismo (*Polymorphism*):** capacità di trattare istanze di classi diverse in modo uniforme attraverso un tipo o interfaccia comune.

0.0.2 2. Codice: Esempio Comparativo: Dal Vettore C all'Oggetto Java

Per apprezzare concretamente i concetti di T02, confrontiamo il frammento C difettoso mostrato dal docente con la sua trasposizione nativa ad oggetti in Java.

Il Problema in C (Codice procedurale insicuro)

```
1 // C procedurale: il dato è nudo e vulnerabile
2 int vect[20];
3 void init() { /* ... */ }
4
5 int main() {
6     vect[25] = 999; // Corruzione di memoria silenziosa! Nessun errore a runtime.
7     return 0;
8 }
```

La Soluzione ad Oggetti in Java (Incapsulamento, Information Hiding, Costruttore) In Java, l'oggetto controlla se stesso. Non esiste memoria non inizializzata e l'accesso fuori limite lancia un'eccezione esplicita a runtime:

```
1 public class SafeVector {
2     // 1. Information Hiding: il dato interno è inaccessibile all'esterno
3     private final int[] elements;
4
5     // 2. Garanzia di Inizializzazione: il costruttore viene eseguito all'atto del 'new'
6     public SafeVector(int size) {
7         if (size <= 0) {
8             throw new IllegalArgumentException("La dimensione deve essere positiva");
9         }
10        this.elements = new int[size]; // Nello Heap, azzerato di default a 0
11    }
12
13    // 3. Controllo dei confini e integrità: interfaccia controllata
14    public void set(int index, int value) {
15        // La JVM effettua l'ArrayBoundsCheck automatico, impedendo corruzioni di memoria
16        this.elements[index] = value;
17    }
18
19    public int get(int index) {
20        return this.elements[index];
21    }
22
23    public int size() {
24        return this.elements.length;
25    }
26 }
```

Punto Importante per l'Esame

Takeaway per l'esame: - Una classe non alloca memoria per lo stato fino alla chiamata a new (che alloca l'oggetto nello **Heap**). - L'Information Hiding si realizza marcando i campi di istanza come private. - I 4 pilastri dell'OOP (*Encapsulation, Abstraction, Inheritance, Polymorphism*) sono il filo conduttore dell'intero corso.

Nota di Studio

Il prossimo blocco è la **Lezione 3 / Slide T03: Building, Running and Deployment in Java**, in cui studieremo: - Il modello di esecuzione Java: **Sorgente (.java)** → **Bytecode (.class)** → **JVM / JIT**. - I comandi CLI fondamentali: javac, java, jar, javadoc. - I concetti di **Classpath** (-cp) e gestione delle dipendenze. - **Apache Maven**: cicli di vita (*compile, test, package, install*), struttura directory standard (src/main/java, src/test/java) e il pom.xml.

Dimmi quando sei pronto per passare a T03!